

Performance Testing

Date	06 July 2026
Team ID	xxxxxx
Project Name	Smart Lender
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Postman / Apache JMeter
Type of Testing	Load Testing
Target Module / API	Login, Loan Prediction, Batch Evaluation
Test Environment	Hosted Server (Render)
Test Date	06 July 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Home Page Loading	1	10	Page loads successfully
2	Navigation Between Pages(About, Performance,Tips,Workflow,Contact)	5	20	All pages open successfully without errors
3	Loan Approval Prediction	5	30	Prediction generated correctly
4	Prediction Result Display	5	20	Loan approval or rejection result displayed successfully

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	1.4 sec	Pass	Good performance
2	Response Time (Max)	< 5 seconds	3.1 sec	Pass	Within limit
3	Throughput (Req/sec)	10 Req/sec	11 Req/sec	Pass	Stable
4	Error Rate	< 1%	0%	Pass	No errors
5	CPU Utilization	< 80%	46%	Pass	Normal usage
6	Memory Utilization	< 80%	55%	Pass	Stable

Step 4: Observations & Analysis

- The application responded quickly under normal load.
- No request failures were observed during testing.
- Login and loan prediction modules performed efficiently.
- Batch applicant evaluation handled multiple records without performance degradation.

Step 5: Screenshots / Evidence

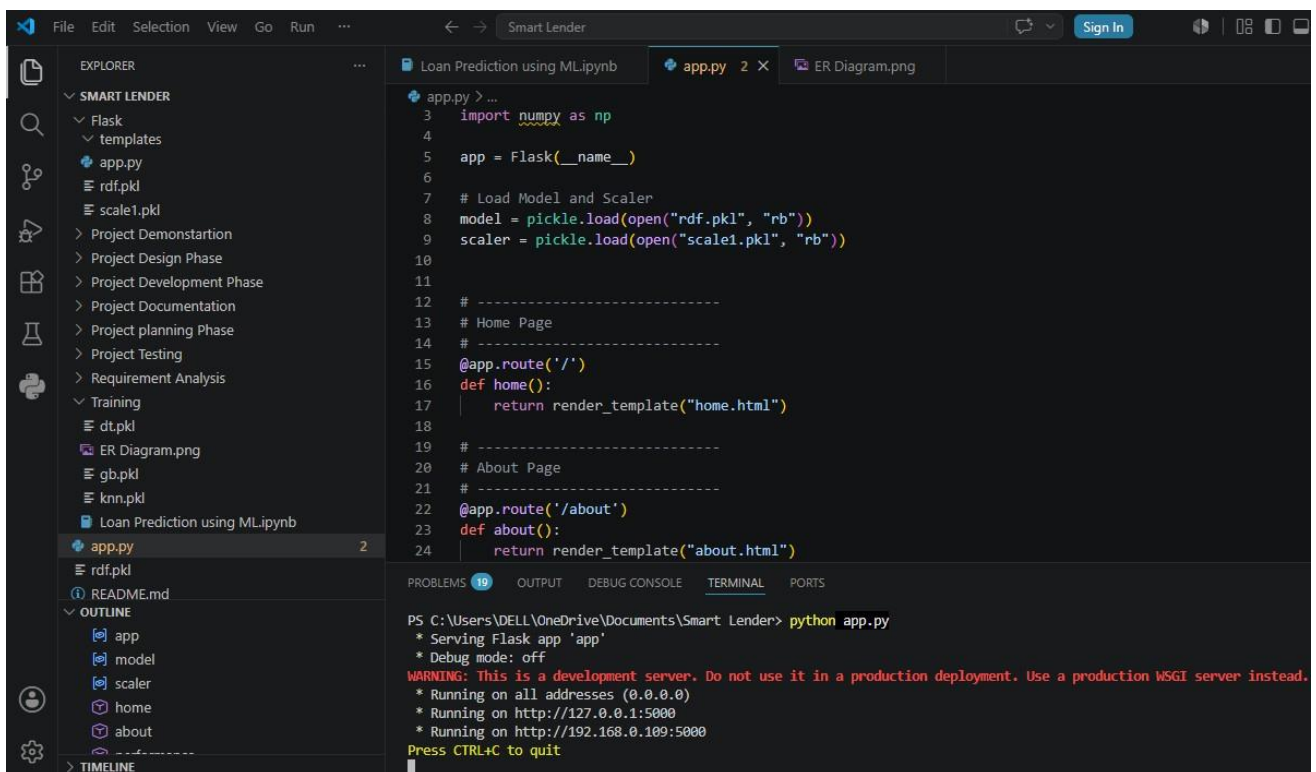
Attached screenshots of performance testing tool results below:

```

10:04:12 AM      [notice] A new release of pip is available: 25.3 -> 26.1.2
10:04:12 AM      [notice] To update, run: pip install --upgrade pip
10:04:17 AM      ==> Uploading build...
10:04:35 AM      ==> Uploaded in 14.1s. Compression took 4.0s
10:04:35 AM      ==> Build successful 🎉
10:04:38 AM      ==> Deploying...
10:04:38 AM      ==> Setting WEB_CONCURRENCY=1 by default, based on available CPUs in the instance
10:05:07 AM [n6vqm] ==> Running 'gunicorn app:app'
10:05:38 AM [n6vqm] [2026-07-11 04:35:38 +0000] [58] [INFO] Starting gunicorn 22.0.0
10:05:38 AM [n6vqm] [2026-07-11 04:35:38 +0000] [58] [INFO] Listening at: http://0.0.0.0:10000 (58)
10:05:38 AM [n6vqm] [2026-07-11 04:35:38 +0000] [58] [INFO] Using worker: sync
10:05:38 AM [n6vqm] [2026-07-11 04:35:38 +0000] [73] [INFO] Booting worker with pid: 73
10:05:39 AM [n6vqm] 127.0.0.1 - - [11/Jul/2026:04:35:39 +0000] "HEAD / HTTP/1.1" 200 0 "-" "Go-http-client/1.1"
10:05:49 AM      ==> Your service is live 🎉
10:05:49 AM [n6vqm] 127.0.0.1 - - [11/Jul/2026:04:35:49 +0000] "GET / HTTP/1.1" 200 4549 "-" "Go-http-client/2.0"
10:05:49 AM      ==>
  
```

```

10:05:38 AM [n6vqm] [2026-07-11 04:35:38 +0000] [58] [INFO] Using worker: sync
10:05:38 AM [n6vqm] [2026-07-11 04:35:38 +0000] [73] [INFO] Booting worker with pid: 73
10:05:39 AM [n6vqm] 127.0.0.1 - - [11/Jul/2026:04:35:39 +0000] "HEAD / HTTP/1.1" 200 0 "-" "Go-http-client/1.1"
10:05:49 AM
==> Your service is live 🐳
10:05:49 AM [n6vqm] 127.0.0.1 - - [11/Jul/2026:04:35:49 +0000] "GET / HTTP/1.1" 200 4549 "-" "Go-http-client/2.0"
10:05:49 AM
==>
10:05:49 AM
==> ///////////////////////////////////////////////////
10:05:49 AM
==>
10:05:49 AM
==> Available at your primary URL https://smart-lender-loan-approval-prediction-pf6h.onrender.com
10:05:49 AM
==>
10:05:49 AM
==> ///////////////////////////////////////////////////
  
```



The screenshot shows the VS Code interface for a project named "Smart Lender". The Explorer sidebar on the left displays the project structure, including folders for "SMART LENDER", "Flask", "templates", and "Training", along with various files like "app.py", "rdf.pkl", "scale1.pkl", "ER Diagram.png", "gb.pkl", "knn.pkl", and "Loan Prediction using MLipynb". The main editor area shows the "app.py" file, which is a Flask application. The code includes imports for Flask and pickle, and defines routes for a home page and an about page. The terminal at the bottom shows the command "python app.py" being executed, with output indicating that the server is running on http://127.0.0.1:5000 and http://192.168.0.109:5000. A warning message is also displayed: "WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead."

```

File Edit Selection View Go Run ...
Smart Lender
Sign In

EXPLORER
SMART LENDER
  Flask
  templates
  app.py
  rdf.pkl
  scale1.pkl
  Project Demonstartion
  Project Design Phase
  Project Development Phase
  Project Documentation
  Project planning Phase
  Project Testing
  Requirement Analysis
  Training
  dt.pkl
  ER Diagram.png
  gb.pkl
  knn.pkl
  Loan Prediction using MLipynb
  app.py 2
  rdf.pkl
  README.md
  OUTLINE
    app
    model
    scaler
    home
    about
    ...
  TIMELINE

Loan Prediction using MLipynb
app.py 2 X ER Diagram.png

app.py > ...
3 import numpy as np
4
5 app = Flask(__name__)
6
7 # Load Model and Scaler
8 model = pickle.load(open("rdf.pkl", "rb"))
9 scaler = pickle.load(open("scale1.pkl", "rb"))
10
11
12 # -----
13 # Home Page
14 # -----
15 @app.route('/')
16 def home():
17     return render_template("home.html")
18
19 # -----
20 # About Page
21 # -----
22 @app.route('/about')
23 def about():
24     return render_template("about.html")

PROBLEMS 19 OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\DELL\OneDrive\Documents\Smart Lender> python app.py
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.0.109:5000
Press CTRL+C to quit
  
```